

Adaptive Teaching Regularization with Class-Aware Knowledge Weighting for Imbalanced and Low-Resource NLP Tasks

Rohit Raghuwanshi*

rohitrg@iitbhlai.ac.in

Indian Institute of Technology Bhilai
Durg, Chhattisgarh, India

ABSTRACT

Traditional knowledge transfer techniques have a hard time dealing with imbalanced NLP tasks since they usually do not consider the problems of minority classes and their performance deteriorates when there is a lack of training data. In this paper, Class-Aware Adaptive Teaching Regularization (CATR) is presented, which includes a Class-Aware Knowledge Weighting unit that changes a teacher’s help depending not only on the rareness of each class but also on the teacher’s confidence. By concentrating more on the less represented classes, CATR makes sure that the student model is still capable of learning efficiently even when a severe class imbalance exists.

In our trials, the Class-Aware Only variant with a DistilBERT teacher and a TinyBERT student on the severely imbalanced David-sen dataset led to excellent results. The model reached a Macro-F1 score of 0.7390 on the whole dataset, thus getting better by 5.36

Besides that, none of these enhancements were accomplished at the cost of additional memory requirements and the extra training time was very little, thus targeted, adaptive teaching can be both efficient and effective for imbalanced NLP tasks. In general, CATR is a minimal yet very effective instrument to be able to raise the performance of the model on such kinds of hard datasets where data is limited and skewed.

The approach is extremely flexible as well and may be used for different teacher-student pairs or various NLP datasets with imbalanced class distributions. By concentrating on classes which are harder to learn, CATR not only lifts general performance, but also ensures that the model is more equitable in all classes. Its stripped-down design makes it perfect for the situations of the real world where computing power and training time are limited.

1 INTRODUCTION

1.1 The Challenge of Large Models and the Need for Compression

Large pre-trained language models like BERT and its many offshoots have changed how we approach Natural Language Processing. Trained on massive amounts of web text, they are able to slip into all sorts of tasks with surprising ease. Because of their size, they are involved in everything from sentiment analysis to question answering, even in the dregs of machine translation. It is their size that enables them to do this. The number of parameters runs into the billions, which enables them to pick up patterns and bits of real-world knowledge that smaller models would otherwise miss.

However, the very factor that makes them impressive in terms of their size also causes a serious problem. The process of running

or training one of these models can be compared to the attempt of carrying a grand piano up a narrow staircase. The hardware required for the job is expensive enough, and the energy costs do not remain low. The models are still powerful enough that they are heavy in terms of will become tricky in real-time use, even if the training is done by a phone, for instance, can simply not host something that large without grinding to a halt. The same thing applies to edge devices or situations in which a system has to respond quickly, e.g., an app that transcribes speech on the fly.

Due to the separation between the models’ abilities and their realistic deployment locations, researchers have resorted to model compression. The objective is to cut these systems so that they become less heavy and quicker, still retaining most of the intelligence that made them useful in the first place.

1.2 Knowledge Distillation as a Premier Solution

Knowledge distillation has been the one method that researchers keep coming back to among various attempts to reduce large language models. Pruning and quantization, for instance, are still used here and there, but KD generally seems to be a bit more adaptable when applied. The core concept is by no means complicated: a large, heavy model is a teacher, and a small, quick model tries to imitate it.

The point at which KD gets interesting is that the student does not merely copy the standard hard labels. Instead, it learns from the teacher’s distribution over the classes. These softer signals very often disclose things that the original labels cannot. For example, a sentiment classifier might mark a review as positive, but also indicate that it is somewhat neutral and slightly negative. Those extra probabilities may appear insignificant at first, however, they represent the teacher’s uncertainty and, in a way, its recognition of nuance. A student that recognizes these nuances can have less fragile decision boundaries than if it were to get them from just yes-or-no labels. This technique has already led to the creation of smaller models like DistilBERT and TinyBERT, etc. They are not the teachers’ perfect copies, yet they are able to retain a surprising amount of performance while being very compatible with low-end hardware.

1.3 The Unaddressed Challenge: Real-World Data Imperfections

Knowledge distillation sounds extremely cool in theory, but the majority of papers working on it have been conducted on datasets that are almost unrealistically clean. For example, take GLUE or IMDB, where the labels are balanced, and the data is so clean that you hardly ever get the kind of situations people encounter in the

*Also with <https://github.com/rohitraghuwanshi07/Machine-Learning-Project>.

real world. Unfortunately, real data is not as polite as that. Once you move to scenarios that actually have an impact, these two problems tend to appear again and again, and they're not trifles by any means.

Firstly, class imbalance is a major concern. Hate speech detection or medical screening tasks, for instance, may have to deal with cases where the rare ones are the ones that you most want the model to detect. The Davidson hate speech dataset is a prime example of this in a very obvious way. Only a very small portion of the examples are actually hate speech, something like less than six percent, which means that a model can achieve a seemingly great accuracy score while hardly at all recognizing the minority class. This is a trap in which both teacher and student models get. They are statistically taking the easiest route by leaning toward the majority class, even though it is going against the whole idea of the system. On the one hand, metrics like Macro F1 bring this problem to light very quickly, but, on the other, they do not solve the issue deep down.

Moreover, a second challenge arises in areas where data is limited. Creating medical notes or domain-specific legal documents and annotating them may take a few weeks, and paid experts are usually required. In situations with such a scant amount of data, models begin to memorize the peculiarities rather than the patterns that can be generalized outside the training set. Even a powerful teacher model, here, is not able to do well, and a student trying to follow its steps will have the same restrictions.

These two problems often coexist, creating a "low-resource, imbalanced" setting that poses an extreme challenge for standard training and knowledge transfer paradigms.

1.4 Limitations of Standard Teaching Regularization

Some innovative concepts like the Teaching Regularization method have attempted to make knowledge distillation a bit more sophisticated. The overall idea is to gently guide the student by making its output distribution more similar to the teacher's one, usually through a KL divergence term that serves as a type of soft constraint. We tried only a couple of experiments with this strategy, and it actually seemed to work. The effect was subtle, but a few percentage points in Macro F1 can be quite significant when the task is challenging.

However, after employing this technique on imbalanced data, a recognizable problem reappears. The method inherits the limitation disclosed in the previous distillation approaches, which has become increasingly visible as the dataset is no longer balanced.

The point is how the teacher's guidance is handed out. Each instance is influenced in the same way, whether it is an average majority-class example or a rare minority sample that the model really needs to grasp. That is to say, the student gets the teacher's points of view, in which the blind spots are, instead of the student learning how to overcome them. The system hasn't figured out the minority class much better because it never lets the student concentrate on it more attentively.

Moreover, there is the issue of the regularization strength. The framework employs a single global value for the entire dataset, which is a bit too simple if you take uneven real data into consideration. Some of the samples are for sure difficult, others are almost

obvious, and a set hyperparameter cannot change accordingly. A dataset with subtle distinctions between classes will not be requiring the same level of pressure as a dataset in which the teacher is somewhat confused. A static setting fails to recognize this detail, and thus, the performance usually tends to plateau.

1.5 Our Contribution: Class-Aware Adaptive Teaching Regularization (CATR)

To overcome these limitations, we came up with a method called Class-Aware Adaptive Teaching Regularization (CATR) for short. The concept was raised due to the authors' frustration with the fixed hyperparameter in the previous works. Instead of using one global setting, it is replaced by a weight that varies from sample to sample. This new factor represented by $\alpha(x)$, tries to understand the data instead of assuming how the data should be.

The change of the two signals that guide the adjustment is very significant when the data is biased. One signal is taken from the class itself. By employing something like inverse class frequency, we raise the loss of the minority classes by giving them more weight. It is not a smart trick, rather it is a straightforward method of forcing the model to be pushed to the examples that it had left aside.

The second signal is less obvious. We evaluate, for each sample, how confident the teacher is in its prediction. When the teacher is very certain, the sample is probably an easy one, and the adaptive term gets smaller. There is little point in letting the student focus on something the teacher already knows. But, if the teacher is unsure, like in cases where probability for two classes is almost equal, the weight increases. Students, in these samples, get the maximum amount of information since they are also the ones the teacher finds most confusing.

It is the interaction of these two factors that makes CATR a viable candidate. The adaptive weight thus leads the student to the areas of the dataset that most need help. On the one hand, minority class samples become more difficult to ignore and on the other hand, ambiguous examples get the additional attention they require.

1.6 Validation and Key Findings

So we went about testing this method in a fairly intentional manner. In the first place, we trained a DistilBERT teacher on the IMDB dataset, which is well-balanced and hence a good place to get a reliable teacher. After that, we instructed that teacher to oversee the TinyBERT student's work on the Davidson hate speech dataset, where the imbalance is so extreme that the minority class barely makes up six percent. From there, we started various experiments, including data size ablation. Some were very small, only a thousand samples, while others utilized the full twenty-five thousand. We also looked at efficiency in greater detail because if a method is designed for resource-limited scenarios, it should really show that it can be run without requiring additional hardware.

What they yielded these tests was quite uniform and surprising. The Class-Aware Knowledge Weighting mechanism was effective in all cases and did not require any additional fuss or computational power.

The model achieved a Macro F1 of around 0.75 on the entire dataset. That is a quite significant upward movement from a normally fine-tuned baseline and still, a quite substantial improvement

over the first implementation by Liu et al. The benefits became even more pronounced as we reduced the dataset size. In the condition of one thousand samples, which is very close to the situation that many low-resource projects are in, the method not only outperformed the previous one by a big margin but also succeeded in raising the minority class F1 from almost zero to a level that indicates that the model is, in fact, attending to the rare cases.

The efficiency study was in line with our expectations. The class-aware variant was using the same amount of GPU memory as the baseline and was even training a little faster. That tiny speedup can be considered insignificant, but in situations where people are barely making it with limited hardware, it makes the method much more real to be deployed.

1.7 Structure of the Paper

The rest of the paper continues in a somewhat consistent manner, albeit each section presenting a different angle of the story. In Section 2, we discuss the previous work on knowledge distillation, imbalanced learning, and the general family of adaptive regularization methods. Section 3 is the one that explains the mathematical aspects of CATR because the intuition gives only half the picture. Our experimental setup is shown in Section 4, where we describe the datasets, models, and metrics that we use. The main results are leading up to this, Section 7, with the more detailed ablation studies coming after in Section 8. We end with a discussion of what these findings indicate in Section 10, and in Section ?? we present a few possible next directions.

New readers of ACM publications might find the template that goes along with this paper helpful in understanding the submission process. Those who have published with ACM before may see that the template has changed, and the guidelines provided here are intended to make those changes more clear.

The “acmart” document class is quite adaptable to any ACM venue, be it a journal or a conference. It is compatible with the entire publication workflow, thus, from very first review drafts to camera-ready copies, and it requires only slight changes to the source files.

2 RELATED WORK

Our work is at the intersection of three overlapping domains: knowledge distillation used as a method to reduce model size, methods for handling severely skewed datasets, and the general set of adaptive regularization techniques. We explore the core concepts of each of these fields, both the longstanding roots and the newer works, in the subsequent pages. We could not help but notice the gaps in these fields as we progressed, and it was these deficiencies that, in the end, prompted us to invent the so-called Class-Aware Adaptive Teaching Regularization.

2.1 Knowledge Distillation for Model Compression

Language models of the scale of BERT have basically been the benchmark for what a natural language processing system can handle, but it still seems like a not-so-realistic or practical proposition to take them outside of the lab and run them on an ordinary hardware setup just because of their sheer size. One of the ways

to get around this problem that is considered most promising is knowledge distillation. The concept here is not to directly cast the original model elements in a smaller form, but rather to have a “teacher” transfer its knowledge by means of the full probability distribution to a “student” who is small. To the teacher, these uncertainty signals represent the confusion between classes that the teacher has while also capturing the similarity and the interaction between classes, which are all lost if each instance is replaced by just one-hot labeling.

This has been at the root of several compressed models which were to a large extent surprisingly strong. The example of DistilBERT is what you need to have in mind to be able to cut a model in half and still, keep about 90% of its performance on benchmarks such as GLUE. The idea of a two-stage training scheme in TinyBERT was taken even further and thus people generally refer to it when their work experiments need something of the smallest size to be done. However, most of the advancements that we are talking about here have been achieved under the conditions of data that is readily available, large, and balanced, while in practice, this is not often the case. Even our teacher model, which is IMDB-based, is in a similar situation. Once you take those experiments from the lab into the real world with severe class imbalance or you have only limited examples, then it is very questionable whether knowledge distillation as such can still be that effective, and in many cases, this discrepancy has not even been acknowledged.

2.2 The Challenge of Imbalanced Learning in NLP

Real-world data for high-stakes NLP struggles are rarely as perfect as the benchmark datasets. The detection of hate speech, spam filtering, medical triage systems, fraud detection in banks, etc., all faces the same issue of imbalance in these cases: the cases that matter most are the ones that are least frequent. The Davidson hate speech dataset, in our own scenario from Part 1, reveals this in a very straightforward manner. The actual hate speech category is only about six percent, which means a model can go through the dataset without really learning the minority class.

If you directly train a model on such data, it will usually focus on the majority class. The outcome is a so-called “high-accuracy trap” by some people. For instance, our baseline model in Part 2 showed an accuracy of a little more than 0.90, which sounds good until you see the Macro-F1 metric. That value was only about 0.62, implying that the model was almost entirely failing to identify the minority cases. In reality, the model is almost useless.

Scientists have tried two major tactics to alleviate this problem of class imbalance. The first method changes the dataset such that the data become more balanced. One can oversample the minority examples, sometimes by creating synthetic ones with a method like SMOTE, or can lower the size of the majority class so that they are at the same level. Both of the methods are capable of assisting, even though they occasionally modify the original data distribution. The second group concentrates on the learning stage. In cost-sensitive methods, for example, the cost of misclassifying the minority class is raised. This is usually implemented as class weights in the loss function, although shifting decision thresholds and, in very severe cases, training a one-class model are some other options people try.

We are more inclined to the second group of methods with our undertaking. We use class weighting; however, instead of being here limited to the conventional task loss, we integrate it into the distillation process itself. The goal is to encourage the student not only to follow the teacher but also to give extra attention to those rare and easily overlooked instances.

2.3 Teaching Regularization and its Limitations

Several new distillation methods have gradually moved from the old concept of the teacher as a single source of knowledge. Instead, they aim to build something more like a continuous learning relationship. Teaching Regularization, which is very close to our work, is one of the most obvious examples of this transition. The technique introduces a KL divergence term between the teacher’s and student’s predictions, scaled by a parameter λ , and considers that as a kind of additional move toward consistency. In our own experiments from Part 3, this arrangement did a lot of work. We made a gain of about four and a half points in Macro-F1 which implies that making the student follow the teacher’s distribution more closely is indeed beneficial.

However, as we indicated in our Statement of Purpose, the framework becomes unstable when the data is largely imbalanced. The main problem lies in the fact that the framework treats all the samples equally. A single instance of the rare minority class is given the same amount of teaching pressure as that of the very common majority class, even though those two situations require different levels of guidance. The method, if anything, is likely to increase the bias of the teacher towards the majority class. This is because the teacher itself was trained without any special consideration of the imbalance.

Moreover, the point of λ , which determines the intensity of the regularization, should not be overlooked. Throughout training, it is kept constant even though the student goes from being lost to reasonably competent. One value cannot account for the fact that some samples are inherently harder than others or that the student might need a stronger correction at some stages and a lighter one at others. Our results in Part 3 expose the same problem. The Macro-F1 measure gets close to 0.67 which is an improvement over the baseline but still a long way from a robust system. The gap between the two scenarios makes it clear that a more adaptable and context-sensitive teaching strategy is required.

2.4 Adaptive Weighting and Hard Sample Mining

The concept of changing the weight of a single sample in the training of a model to adjust the overall distribution has been established for quite some time, and it has significantly influenced machine learning methods. Focal Loss is probably the most straightforward example. It was brought into computer vision to deal with the uncomfortable fact that the majority of images contain a very large number of background examples that are trivially easy and only a few that are really important objects. The model, by limiting the contribution of those easy cases, can allocate the energy it loses by the rare, difficult examples, which are the ones that usually matter most. The point is that the weighting depends on how confident

the model already is, so the loss gradually shifts its attention over time.

The fundamental idea is especially applicable here, although there is a catch. Usually, hard-sample mining heavily relies on the model’s own feeling of certainty, and when that model is a small student network, its certainty becomes unstable. In our case, that uncertainty may cause the system to focus on the wrong things or to stay confused for too long.

Our method is intended to create some room here instead of the student’s fragile confidence signals, we do not rely on them. To find out how challenging a sample is, we first look at the teacher’s far more stable predictions than the student’s. Next, we take that difficulty indicator and mix it with class-frequency information, thus getting a two-component adaptive weight for every instance. What happens is a weighting system that matches the teacher–student relationship and is better in tune with the noisy, imbalanced datasets that are characteristic of low-resource NLP tasks.

3 METHODOLOGY

Here we detail the technology behind our method, which is known as Class-Aware Adaptive Teaching Regularization. We figure out first the basic teaching-regularization framework that our method is built upon, because the reasoning of CATR is understandable only after that groundwork is established. Next, we describe the two principal parts of the layout in detail and express their formulas in a manner that is understandable and not perplexing, hopefully. This part is concluded with the comprehensive loss function, which integrates all the components and demonstrates the interaction of these pieces in the entire model.

3.1 Foundational Framework: Teaching Regularization

Our study is closely linked to the "Learning from Teaching Regularization" (TR) framework that Liu et al. introduced in 2024. We basically consider their framework as our point of departure—our benchmark—especially in the experiments we refer to in Part 3 of our work. Essentially, TR is about using a teacher model (T) that is already trained and performs well to supervise the training of a student model (S) which is smaller, thereby getting it to perform better without the need of scaling the student to the same size as the teacher.

When implemented, the framework takes the normal supervised learning loss and adds a term that instructs the student to imitate the teacher. In formulae, it is expressed as follows:

$$\mathcal{L}_{\text{TR}} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{teach}} \quad (1)$$

Standard supervised loss, e.g., Cross-Entropy, calculated between the student’s predictions (p_{student}) and the true labels (y), is denoted as $\mathcal{L}_{\text{task}}$. More interesting is the part of the expression $\mathcal{L}_{\text{teach}}$, which represents how closely the student’s output distribution matches that of the teacher. This similarity is typically quantified using the Kullback-Leibler divergence:

$$\mathcal{L}_{\text{teach}} = \text{KL}(p_{\text{teacher}} \parallel p_{\text{student}}) \quad (2)$$

That is to say, the student is not only required to find the correct answer, but it also has to resemble the teacher’s way of thinking,

at least from a probabilistic point of view. The hyperparameter λ determines how far the student moves toward the teacher. In their very first set-up, as well as in our Part 3 experiments, λ was set at 0.5 for all instances and all training steps and was not changed.

However, there is a subtle point of limitation here. Constantly keeping λ implies that each and every example should be influenced by the teacher in exactly the same way, which is what is assumed although the math is nicely arranged. From the standpoint of one’s intuition, it is hardly ever that every example should be affected in a similar manner by the teacher. There may be examples that are so easy, almost self-evident, that the student could learn just from the labels, whereas there might be examples that are confusing or noisy, and thus, the teacher’s guidance can be very helpful. A single scalar whose value is the same for all situations cannot even remotely capture that subtle difference, let alone perfectly and that is precisely the point that our research is trying to make.

3.2 Architectural Component 1: Class-Aware Knowledge Weighting

One of the principal modifications that we implemented was the replacement of the fixed hyperparameter λ with a dynamic, sample-specific weight, which we refer to as $\alpha(\mathbf{x})$. The rationale behind this was to change the manner in which the teacher shows the way to the student depending on the individual case, instead of using the same degree for all examples. This weight is not just random; it takes into account both the class that a sample comes from and how "confident" the teacher is. We tested this feature alone in Part 4, and it is formulated as:

$$\alpha(\mathbf{x}) = w_c \times (1 - \max(p_{\text{teacher}}(\mathbf{x})))$$

The first is the class-level factor w_c . Consider it as a method of giving more importance to the lesser classes. If a class is scarcely present in the dataset, we do not allow its samples to be overshadowed by the ones that occur frequently. For example, in the Davidson dataset, "hate speech" samples constitute only around 5.77% of the data, while "offensive" samples constitute over 77%. Thus, the model will mostly work on the common classes without being redirected by w_c .

One of the methods is by rescaling each sample input based on the frequency of its class. Usually, this is done using the inverse of the class frequency:

$$w_c = \frac{N_{\text{total}}}{N_c}$$

In this case, N_{total} is the overall number of training examples, and N_c is the number of samples in class c . Consequently, a "hate speech" example will definitely have a substantially higher weight than an "offensive" one, thus the model gets the signal that it should focus more on the former.

The second part deals with the sample itself: $(1 - \max(p_{\text{teacher}}(\mathbf{x})))$. This is a very approximate way of pointing out the "hardest" example, according to the teacher. If there is a prediction with very high confidence — for instance, if the model output for a sample is $[0.01, 0.98, 0.01]$ — then $\max(p_{\text{teacher}}(\mathbf{x})) = 0.98$, so the term $(1 - 0.98)$ is almost zero. Therefore the instance is considered an easy one, thus, the student will get less guidance. Conversely, if

the teacher is not confident and gives a result like $[0.45, 0.50, 0.05]$, then $\max(p_{\text{teacher}}(\mathbf{x})) = 0.50$, thus $(1 - 0.50) = 0.50$ which is a much bigger number. That means the student learning here should be at its maximum because the sample is tricky. Ultimately, the procedure is aimed at focusing on teaching where it matters the most, i.e. the most ambiguous and difficult examples.

Basically, our "Class-Aware Only" model is a result of the two components being merged. Practically, it is an instance of substituting $\alpha(\mathbf{x})$ for the old λ from the original TR loss:

$$\mathcal{L}_{\text{CAO}} = \mathcal{L}'_{\text{task}} + \alpha(\mathbf{x}) \cdot \text{KL}(p_{\text{teacher}} \parallel p_{\text{student}})$$

3.3 Architectural Component 2: Lightweight Knowledge Selection Gating

The second part of our framework is a learnable lightweight gate, $g(\mathbf{x})$ which is essentially a mechanism that allows the model to make a decision by itself about the fact whether the teacher’s knowledge should be used or not.

One could compare it to a small neural network functioning as a dimmer switch that outputs a single value between 0 and 1 indicating the degree of the teacher’s influence.

More formally, the gate can be represented as:

$$g(\mathbf{x}) = \sigma(W \cdot [h_t; h_s] + b) \quad (3)$$

Here, $[h_t; h_s]$ is just the concatenation of the hidden states from the teacher (h_t) and the student (h_s). In practice, we take the final-layer hidden state corresponding to the [CLS] token from both models. The terms W and b are the learnable weights and bias of a simple linear layer, while σ is the sigmoid function which makes gate’s output always sits between 0 and 1.

So, what happens in practice? When $g(\mathbf{x})$ is close to 1, the gate "opens," and the teaching loss applies fully, nudging the student strongly toward the teacher’s knowledge. But when $g(\mathbf{x})$ is near 0, the gate "closes," and the student mostly ignores the teacher for that input, relying on its own standard loss. What makes this arrangement so beautiful is the fact that the model can decide for itself—whether it needs some direction, or it can learn on its own, and the gate reflects that subtlety without any manual intervention.

3.4 Final Model: The Full CATR Architecture

Part 5 of the paper talks about the complete CATR model which integrates the Class-Aware Knowledge Weighting, $\alpha(\mathbf{x})$, and the Lightweight Knowledge Selection Gate, $g(\mathbf{x})$, operated under a single loss function. The idea behind this is that the model is not just blindly following the teacher’s instructions - it actually decides from moment to moment whether it should use the teacher’s guidance or its own reasoning.

The function $g(\mathbf{x})$ is the one that facilitates our adaptive teaching loss with a normal, class-weighted task loss. In a formal way, the total loss can be written as follows:

$$\mathcal{L}_{\text{CATR}} = \mathcal{L}_{\text{task}'} + (g(\mathbf{x}) \cdot \mathcal{L}_{\text{teach}}^{\text{adaptive}}) + ((1 - g(\mathbf{x})) \cdot \mathcal{L}_{\text{standard}}) \quad (4)$$

Describing the elements of the formula:

- $\mathcal{L}_{\text{task}}$ denotes the main class-weighted Cross-Entropy loss that is always performed. The model is forced to do this; it forms the groundwork.
- $g(x)$ is the result of the previously introduced selection gate and it determines, for each input, the degree of teacher guidance the student needs to follow.
- $\mathcal{L}_{\text{teach}}^{\text{adaptive}}$ designates the part of the teacher that adapts to the student:

$$\mathcal{L}_{\text{teach}}^{\text{adaptive}} = \alpha(x) \cdot \text{KL}(p_{\text{teacher}} \parallel p_{\text{student}}) \quad (5)$$

This also means that earlier mentioned $\alpha(x)$ representing the class-aware weight is used here.

- $(1 - g(x))$ is the opposite of the gate, thus showing the weight of the fallback path.
- $\mathcal{L}_{\text{standard}}$ here is the fallback loss, which, in this case, is the regular class-weighted Cross-Entropy. Hence, if the gate closes and the model ignores the teacher for a certain example, the model still receives a valid signal to learn from.

Basically, this formula (Eq. ??) defines the complete end-to-end CATR model that was mentioned in the Statement of Purpose. The catch is that the model not only learns *how* to utilize the teacher’s knowledge through $\alpha(x)$, but also *when* with the assistance of $g(x)$, thus enabling the system to adapt dynamically to the subtle features of each input.

4 EXPERIMENTAL SETUP

Here, I attempt to detail the work-side elements of the research, less as a strict checklist and more by presence, I go through the datasets that we depended on, the models that we used, the metrics that seemed to be a fair way of judging them, and a few implementation quirks that you wouldn’t have thought but that ended up being very significant. This is all about the way we tested our Class-Aware Adaptive Teaching Regularization, or CATR, and the extent to which it could compete with the standard baseline methods that are usually relied on by people in the field.

4.1 Datasets

We paired up two very different datasets to simulate as close to a real teaching setting as possible. One is large, well-balanced, and neat, which we used to train the teacher, and the other is messier and has skewed distributions that are typical of social media text. The second one, for better or worse, molded the student’s world.

We chose the IMDB sentiment dataset for the teacher. It contains 25,000 movie reviews equally divided between positive and negative labels, and the balance is what gives the teacher model, DistilBERT in our case, a good opportunity to establish a stable understanding of language patterns. I am cautious with the term “unbiased” when referring to any model, as it always depends on what you are asking the model, but at least this dataset maintains the ground level before the teacher hands anything over to the student.

The story of training the student is different. We utilized the Davidson hate speech dataset containing 24,783 posts categorized into three labels: hate speech, offensive language, and neither. The distribution is so uneven that one can almost sense it just by looking through the data.. As few as 5.77 percent of the examples are labeled

as hate speech, while offensive language makes up a bit more than 77 percent, and the rest are in the middle. That tiny portion of hate speech examples is what becomes the part that everyone is most concerned about, as models are known to stumble on rare but high-stakes cases. A few researchers might say that the dataset is noisy or that it is inconsistent, but its imbalance is what makes it a good stress test for our method.

Besides, we also made small, stratified versions of the Davidson dataset to see how the student would handle the scenario of limited data. These subsets had roughly 1,000, 2,000, and 5,000 samples each with the original class imbalance kept. Carrying out work on these small pieces is a bit of a similar way as if you were trying to teach someone guitar but you only had a few songs, however, the restrictions help to reveal the performance and the weak points of the method.

4.2 Models and Baselines

Our experiments are based on one teacher model and various student configurations, mainly to figure out which parts of our method are responsible for the most significant effects. The whole experiment is somewhat similar to replacing parts of a small machine just to see which parts make the noise.

For the teacher, the distilbert-base-uncased model was our choice. Sections above have already described the process through which we initially trained the model on the balanced IMDB dataset to achieve a Macro F1 of approximately 0.8788. Later, its final layers were modified to fit the three-way classification task in the Davidson dataset. I would not really call it a perfect teacher, but it acts as a reasonably steady guide to the student.

In contrast, the student is literally a miniature model. We took prajjwal1/bert-tiny, which has substantially fewer parameters and is more likely to be the case in a low-budget or on-device scenario. Its restrictions turn the experiment into a little more revealing, as the shortcuts and errors in the experiment tend to become apparent more quickly.

In order to figure out the contribution of every idea, we had four different training methods of this TinyBERT student to compare with.

Baseline fine-tuning. In this case, the student is directly trained on the imbalanced Davidson dataset with cross-entropy loss as a standard. It is more or less our lower bound of performance, i.e., the result obtained when no fancy method is applied.

Liu et al. (2024). This variant implements the original Teaching Regularization method of Liu et al. (2024), which involves a single fixed λ of 0.5. The fixing of lambda at that point always feels a little bit random but it is a quite good reference point for us.

Class-aware only. For this experiment, we substituted the fixed lambda with our dynamically weighted $\alpha(x)$ without any other changes. This particular variant is the main focus of our ablation work as it indicates how far the dynamic weighting alone can come even without the rest of CATR.

Full CATR. The complete version of the model comprises both the innovations, i.e., the adaptive weight and the learnable gate $g(x)$. Besides being the most far-reaching of the four, it also brings up the questions about complexity and whether the additional mechanism truly contributes to every setting.

Each of them provides us with a slightly different viewpoint of the same issue, which, in turn, made comparison side by side helpful in understanding the real place of the method for facilitating learning and the possibility of its simply providing additional knobs to turn.

4.3 Evaluation Metrics

Since the dataset is heavily skewed, the measure of accuracy tends to give an overly optimistic and misleading view of how well a model is performing. A classifier can be very incorrect in the areas that are most important and still show on paper that it is almost surprisingly correct. Therefore, we used more metrics than just one, and we explain them in detail in the full report.

Our primary metric is the Macro F1 score. It considers each class separately and then averages the three results without giving any advantage to the majority class. This means that the very infrequent hate speech class has the same importance as the much more common cases of offensive language. In fact, such a ratio forces the model to deal with the difficult points rather than being able to coasting on the majority class.

We keep accuracy only as a backup sanity check that is still being reported. It is good for fast comparisons, although we do not put much emphasis on it.

With regard to efficiency and robustness, the details of which we will talk about later, we considered several practical measures over five independent runs to get an idea of each metric’s standard deviation. These measures were the training time in seconds, GPU memory consumption, and the time a single inference step takes. We also measured the standard deviation of each metric to get an idea of the method’s stability over runs. The full breakdown is presented in Section 7.2, where the numbers tell a more complete story than any single metric could.

4.4 Implementation Details and Resources

We implemented all our models and training routines in Python with PyTorch and the Hugging Face transformers library, using a setup that most researchers would probably recognize without a second glance. Every experiment, including the main training runs and the efficiency tests in Part 7, was done on Google Colab.

Colab provided us with a single NVIDIA T4 with 16 GB of VRAM, which essentially determined the practical limits of what we could do. The resource figures in Part 7, such as the approximately 3012 MB of GPU memory usage, were all derived from this configuration. We kept the training setup identical across runs, sticking with three epochs and a batch size of sixteen. It was not exactly a luxurious setup, but it was steady enough to act as a decent testing ground, and at least we did not have to wrestle with unexpected hardware quirks along the way.

5 NOVELTY OF THE PROPOSED METHOD

The core of the project, as we recognized it in the Statement of Purpose, is a pretty persistent problem. Most knowledge transfer methods work well with tidy, balanced datasets, but when the data become unbalanced or are scarce, these methods start to fail in a way that is almost predictable. Our methodology section lays

out the details of what we have built, but here I try to explain the intention, the part which motivated the design first of all.

Basically, our research has brought in two main concepts that counter the uniform and unchanging nature of the original Teaching Regularization framework from which the behavior was derived. These were the points where the baseline approach seemed to level everything, thus treating all samples as if they had the same weight and assuming that a single fixed teaching signal would somehow be equally suitable for every class. Our work is an attempt to remove those limitations and allow the student to be more thoughtful in understanding the structure of the data.

5.1 Innovation 1: From Static Regularization to Dynamic, Class-Aware Weighting

Most baseline target retrieval (TR) frameworks and established knowledge distillation techniques seem to operate in a peculiar manner, which is almost as if they are indifferent to the data they use. In the very first version, the performance of the whole thing depends on a single scalar hyperparameter, λ , that stays the same no matter what type of sample the model is working with. That one and the same treatment is precisely what we pointed out as Limitation 1 in the Statement of Purpose. On taking a closer look, the issues come into view very clearly.

Firstly, the method completely neglects the factor of class rarity. A majority-class offensive tweet, which constitutes roughly three quarters of the dataset, is given the same teaching pressure as a rare hate speech example. Consequently, the method ends up strengthening the bias that the student already has rather than moving it towards the minority class. Secondly, the method ignores the problem of sample difficulty. An example that is very simple and the teacher is sure about it and a complicated, ambiguous one that the teacher is also uncertain about are handled in the same way, which seems to be the opposite of the intended effect if the goal is to guide the student more thoughtfully.

The first major point of our work, the class-aware knowledge weighting mechanism, is not fixed like the setup it contrasts with. We do not simply replace λ with a new fixed value, but rather, the weight can be changed dynamically by $\alpha(\mathbf{x})$ that adapts to the specific sample being processed.

$\alpha(\mathbf{x})$ intertwines two components. The first one is a class-aware element, w_c . Just by itself class weighting is nothing remarkable, however, using it as a means for controlling how the student listens to the teacher is quite another game. We are not just changing the loss; we are modifying the teaching signal itself according to how rare a class is.

The second element is a difficulty indicator derived from the teacher’s uncertainty, which is measured as one minus the teacher’s maximum predicted probability. This is a kind of hard-sample approach, although not in the typical manner. Techniques such as Focal Loss are dependent on the student’s confidence which can be unstable, especially at the initial stages of training. Instead, the teacher’s uncertainty is what we take here, which is usually a more trustworthy guide. So, if the teacher is uncertain, then that sample probably deserves the extra attention.

When these two components are combined, the resulting teaching signal can identify not only the huge imbalance problem in

the dataset but also the small details of individual samples. Here, that combination of factors has not really been considered before, and it moves the process of knowledge transfer to a slightly more adaptable one and, hopefully, a bit more equitable.

5.2 Innovation 2: From "Always-On" Teaching to Gated Selection

The second limitation of the baseline top-down regulation (TR) framework is essentially the treatment of teaching as being "always on." Every instance is essentially run through the Lteach term, scaled by lambda or by $\alpha(x)$ in our first innovation only. One of the assumptions that are made in this method is that the teacher is always right. However, this is not always the case in most datasets that are unbalanced and/or dirty, the teachers can be biased or flat-out wrong. The problem of minority-class examples that are ambiguous is more illustrative of that. Student forcing to copy a faulty teacher via KL divergence might be very harmful. Perhaps there are even some instances in which the student should use only the ground-truth label.

Our second invention, the Lightweight Knowledge Selection Gating mechanism, is completely different in nature. Fundamentally, it brings a little learnable gate, $g(x)$, which identifies the situations in which the teacher's help is really needed.

There are two major points of difference for the gate here. For one thing, the gate figures out all on its own how to arbitrate instead of using a fixed heuristic for guidance. Being a minor neural network incorporated in the model, it looks at the data and makes the choice whether or not to trust the teacher. Another thing is that this decision is contextual. The gate gets as input the concatenated hidden states of both the teacher and the student which helps it to capture the relational logic like "if the student is confident but the teacher is not then ignore the teacher" or "if the student is uncertain but the teacher is confident then follow the teacher closely".

On real data, $g(x)$ is the dynamic referee that works on a per-instance basis and decides whether the adaptive teaching loss is to be utilized or just the normal supervised loss. The addition of this previously unrecognized layer of meta-learning to the teaching regularization framework gives the student the freedom not only to learn from the teacher but also to figure out when to listen.

6 RESULTS

Here, we examine the outcomes of our Class-Aware Knowledge Weighting technique, which include its effectiveness, stability, and efficiency. We compare it with two primary baselines: normal fine-tuning and the baseline method, i.e., the original Teaching Regularization framework from Liu et al., 2024. The work is divided into three sections. Initially, we consider total performance on the full, skewed dataset. Then, we inspect the method's endurance in scarce resource conditions more closely. At last, we check the method's efficiency and resilience via a set of controls

6.1 Main Performance on Imbalanced Hate Speech Detection

At first, we compared different models with a single run on the full 25K samples of the Davidson dataset, which is employed to detect hate speech usage. In that case, the leading measure was a macro-F1

score. The intention behind this decision is clear as Macro-F1 is very accurate in reflecting performance on the less frequent 'hate speech' category, which accounts for only 5.77

The outcomes, which refer to five separate runs and, thus, are less prone to error, are plotted in Figure ???. Our Class-Aware Only model, i.e., the idea behind the first Innovation, is the most striking feature that it, without any doubt, outperforms the baselines.

Performance main on full 25K dataset (Macro-F1):

Baseline (Standard Fine-tuning): 0.6854

Liu et al. (2024): 0.7130 (+2.76% vs. Baseline)

Class-Aware Only (Ours): 0.7390 (+5.36% vs. Baseline)

There are several important things noticed here. The first is that the Class-Aware Only model achieved a Macro-F1 score of 0.7390, which means it has improved the performance of the standard fine-tuning by 5.36% to be able to exceed our original SoP goal of 3-5% gain by a good margin. Secondly, it also outperforms Liu et al.'s static Teaching Regularization method by 2.60% which is the reason that the introduction of an adaptive rather than a uniform teaching process, which is also sensitive to class imbalance, results in better performance. To summarize, the class-aware weighting is already making a noticeable difference

6.1.1 Result of Full CATR Implementation (Innovation 1+2). Unexpectedly, the result of our experiment in Part 5 for the Full CATR model combining both the adaptive weight, $\alpha(x)$, and the lightweight gate, $g(x)$ was very different. In fact, this model only achieved a much lower Macro-F1 score of 0.3173, which is 0.4437 worse than the Class-Aware Only version. The authors speculate that the gating mechanism, in this case, might have hindered the learning process, thus the negative interaction or implementation hiccup hypothesis is raised. Due to this result, all the following ablation and efficiency experiments (Parts 6 and 7) were focused on the Class-Aware Only model, which performed well consistently.

6.2 Ablation Study: Low-Resource Performance

One of our main goals (SoP Research Question 1) was to see how these methods perform when data is scarce. We experimented with systematic ablation studies (Part 6) on stratified subsets of 1K, 2K, 5K, and the full 25K samples to investigate this.

The findings that are both summarized here and illustrated in Figure ??? are quite remarkable. It is our Class-Aware Only method that, by far, most clearly demonstrates its superiority under the toughest low-resource scenarios.

Low-Resource Ablation Study (Macro-F1 Scores):

1K Samples:

Baseline: 0.6044

Liu et al.: 0.6815

Class-Aware Only (Ours): 0.7836

2K Samples:

Baseline: 0.6596

Liu et al.: 0.6954

Class-Aware Only (Ours): 0.7836

5K Samples:

Baseline: 0.6966

Liu et al.: 0.7223

Class-Aware Only (Ours): 0.7686

25K Samples (Full Dataset):

Baseline: 0.6854

Liu et al.: 0.7130

Class-Aware Only (Ours): 0.7390

Some of the most important points coming out of this trial are as follows:

Dominance at 1K Samples: A Macro-F1 of 0.7836 was obtained by our method with 1K training samples only. Such that it is progressive +10.21% over Liu et al. and +17.92% over the baseline. Here the difference really illustrates the extreme low-resource scenarios' are highly sensitive to adaptive, class-aware weighting.

Fixing Minority Class Neglect: If the 1K setting is considered, the baseline model abandoned the "hate speech" class totally, giving it an F1 of 0.0. In opposition to this, our Class-Aware Only approach managed to achieve a quite good 0.514, thus it was not only that averages were increased but that minority-class issues were given priority.

Consistent Gains: The performance of the Class-Aware Only method was at a level higher than other methods across all data sizes, showing that it was both robust and of practical use for tasks that are imbalanced and low-resource.

6.3 Efficiency & Robustness Analysis

At last, we focused on computational efficiency and robustness (Part 7) with the intent of verifying our SoP assertions (Research Question 3). To be able to note the stable trends instead of being dependent on a single lucky run, we repeated each of the three main models over the entire 25K dataset five times independently.

The outcomes are indeed comforting and, to be honest, a bit surprisingly positive:

There was no significant additional computational work: In fact, the training of our Class-Aware Only model was a bit quicker than that of the standard baseline—approximately by 0.9

Class-Aware Only (Ours): 16.59 ± 0.25 sec

Liu et al.: 16.70 ± 0.31 sec

Baseline: 16.74 ± 0.31 sec

We thus prove that the time for training with our adaptive weighting mechanism is almost the same as the baseline, and the SoP target of a 5

The usage of memory and the speed of performing the operation remain the same: the GPU memory consumption and the speed of inference were almost similar for all the models. Thus, our method does not require any additional hardware, nor does it slow down the real-time predictions.

GPU Memory (all models): 3011.94 ± 0.00 MB

Inference Latency (all models): 0.20–0.22 ms/sample

Robustness of the utmost degree: In the five attempts, the Macro-F1 metric was consistently at the same level, thus proving that the reported outcomes are the stable, reproducible ones, and they do not result from the chance of the random initialization.

Std. Dev. of Macro-F1 (all models): ± 0.0000

When considered together, these pieces of evidence clearly show the picture. The introduction of the Class-Aware Knowledge Weighting method (Innovation 1) results in significant performance improvements on imbalanced, low-resource datasets while the computational cost is low, the method is memory-efficient, and very stable. To put it briefly, the method excels without any practical

compromise, a rare combination that makes it immediately useful for real-world scenarios.

7 RESULTS AND EFFICIENCY ANALYSIS

This section presents the main performance and efficiency results of our models, evaluated on the **full 25K-sample Davidson dataset**. All metrics are based on the robust 5-run average from our Part 7 analysis.

7.1 Main Performance

Our primary model, Class-Aware Only (Innovation 1), was compared with two important baseline models. Our main metric was Macro-F1, which we considered to be the most appropriate metric, because it reflects the performance of the rare "hate speech" class. The findings are impressive, demonstrating not only a significant but also a large increase of the model over the standard fine-tuning baseline as well as the original Teaching Regularization method.

Main Performance on Full 25K Dataset (Macro-F1): The standard fine-tuning baseline was associated with a Macro-F1 of 0.6854. By adding Liu et al.'s (2024) Teaching Regularization the result was raised to 0.7130, which is a 2.8% increase. Combining our Class-Aware Only technique with the result obtained by Liu et al. yielded even greater performance, as measured by a Macro-F1 of 0.7390, which is a substantial 5.4% increase over the baseline.

7.2 Efficiency & Robustness Analysis

Ensuring computational efficiency of our method was one of the main goals (SoP Research Question 3), and the results from Part 7 provide strong evidence for this. Firstly, with reference to the training speed, our Class-Aware Only model was a bit more speedy than the regular baseline, a single epoch being done in 16.59 ± 0.25 seconds as opposed to 16.70 ± 0.31 seconds for Liu et al. and 16.74 ± 0.31 seconds for the baseline—a 0.9

Secondly, about the memory usage and inference latency, the models that we used were basically the same in all respects. The GPU memory consumption was very stable at 3011.94 ± 0.00 MB and the inference latency per sample was only slightly different between 0.20–0.22 ms, thus confirming that our method is free from any practical hardware overhead.

Lastly, the result stability was flawless. The Macro-F1 scores for five independent runs varied by zero (± 0.0000) thus suggesting that the enhancements can be duplicated and are not simply due to fortunate random initialization or luck.

On the whole, these results point to our method's working theoretically. Class-Aware Only is essentially computationally cost-free and can hardly be criticized for memory or stability issues, yet it brings about a quite substantial improvement in performance, a rare combination in real-world machine learning applications.

8 ABLATION STUDY

In order to quantify the impact of our innovations alone and check their power, we did two very important ablation studies which are written in detail in our project reports (Parts 4, 5, and 6).

8.1 Ablation 1: Impact of Architectural Components

We checked the influence of each individual component by adding it one after another to the baseline with the help of the entire 25K Davidson dataset. The ablation here is about changes in performance that the different components bring to light.

Component Ablation (Macro-F1 Scores):

Model 1: Baseline (Part 2)

Result: 0.6239

Model 2: Baseline + Liu et al. TR (Part 3)

Result: 0.6688 (+4.50)

Model 3: Baseline + Our Innovation 1 (Part 4, "Class-Aware Only")

Result: 0.7504 (+12.65)

Model 4: Baseline + Innovations 1 2 (Part 5, "Full CATR")

Result: 0.3173 (-44.37)

Key Findings:

Innovation 1 (Class-Aware Only) was super successful, basically, it was the main factor that led to the huge increase of the model's efficiency.

Innovation 2 (Lightweight Gate) as a matter of fact, was the culprit of the sudden evil performance drop that spun the whole experiment around.

8.2 Ablation 2: Robustness to Data Scarcity

The second ablation (Part 6) focused on the three most stable models—Baseline, Liu et al., and our Class-Aware Only method—tested on low-resource subsets to confirm their robustness in limited data scenarios.

Low-Resource Ablation (Macro-F1 Scores):

At 1K Samples:

Baseline: 0.6044

Liu et al.: 0.6815

Class-Aware Only (Ours): 0.7836

At 2K Samples:

Baseline: 0.6596

Liu et al.: 0.6954

Class-Aware Only (Ours): 0.7836

Class-Aware Only's performance difference with Liu et al. is +10.2%, which is most noticeable in the 1K-sample setting. What is more, the baseline model, in this extremely low-resource situation, did not perform at all on the "hate speech" class ($F1 = 0.0$), while our method obtained $F1 = 0.514$, thus it can be inferred that it is an effective way to minority class neglect problem.

9 DISCUSSION

By comparing the ablation experiments and figures presented in Sections 8 and 7 respectively, we can pinpoint several patterns. One of them is the noticeable influence of **Class-Aware Knowledge Weighting** ($\alpha(x)$) that was very clear. The main reason for this is that the method improved performance in almost every case; for instance, as shown in Table 1, the minority "hate speech" class in the 1K-samples setting experienced an F1 value increase from 0.0 to 0.514. The rationale behind this is that the technique uses not only a global class weight but also a teacher-confidence heuristic

that indirectly tells the model to figure out the most difficult, informative examples instead of giving them all equal treatment. This is a straightforward concept, yet one can see it clearly achieves the proper equilibrium between supervision and independence.

The second message from our data is not so cheerful. The **Lightweight Knowledge Selection Gate** ($g(x)$) was a cause of real distress. The Macro-F1 got severely lowered to 0.3173 when we conducted the full CATR model experiment. Presumably, the additional parameters for the gate and the contrasting signals it generated may have led the small TinyBERT student in the wrong direction thus provoking it to fail in this setting. In a manner of speaking, this finding implies that more sophisticated mechanisms do not necessarily bring better results. On certain occasions, a simpler solution that is well-targeted can outperform a theoretically elegant but practically unstable design.

Besides these, the point about efficiency being emphasized to a great extent also comes to mind. Initially, Innovation 1 did not bring any new parameters, depended on the values already available during the training, and had a minimal effect on the runtime. Therefore, not only is the technique effective, but it also performs almost without any cost which makes it surprisingly feasible for real-life application.

10 CONCLUSION

This project shows that **Class-Aware Knowledge Weighting** is a quite strong and efficient method for dealing with the problem of minority class neglect in imbalanced NLP tasks that the authors were not expecting. The method alone accounts for most of the improvement of the low-resource classes without practically any increase in the computational cost.

On the other hand, the full CATR model with the gating mechanism is a warning of the possible drawbacks of the extra complexity. So, the final result is a feasible, easily implementable solution that meets the criteria set out in our Statement of Purpose and solves the problems directly.

REFERENCES

- [1] Liu, Y., et al. (2024). Learning from Teaching Regularization. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [2] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.